

RÉPUBLIQUE DU BÉNIN

MINISTÈRE DE L'ENSEIGNEMENT SUPÉRIEUR ET DE LA RECHERCHE
SCIENTIFIQUE

UNIVERSITÉ D'ABOMEY-CALAVI (UAC)

*INSTITUT DE FORMATION ET DE RECHERCHE EN INFORMATIQUE
(IFRI)*

MASTER 1 : Big Data — Travaux Pratiques n° 4

Clustering IoT par K-Means MapReduce

Modélisation parallèle sur capteurs distribués — 5 serveurs régionaux

Génie Logiciel

Année académique : 2025-2026

Contents

1 Introduction

Les réseaux de capteurs IoT (*Internet of Things*) génèrent des volumes de données croissants, répartis sur plusieurs sites géographiques. Dans ce contexte, regrouper automatiquement des milliers de capteurs en zones aux comportements similaires constitue un problème central de l'analyse Big Data.

L'algorithme de clustering K-Means est une solution classique à ce problème. Cependant, son exécution séquentielle sur un seul processeur atteint rapidement ses limites lorsque le volume de données augmente. Le paradigme **MapReduce** offre une alternative : en distribuant les calculs sur plusieurs nœuds, il permet de traiter de grands volumes de données tout en conservant la précision de l'algorithme d'origine.

Ce rapport présente l'implémentation et l'évaluation du K-Means MapReduce appliqué à un réseau IoT simulé de cinq serveurs régionaux (Nord, Centre, Sud, Ouest, Est), chacun stockant les mesures de 300 capteurs, soit 1 500 capteurs au total. Les expériences ont été conduites sur une machine à 4 CPU logiques. Le rapport s'organise comme suit : la section 2 modélise le problème et décrit l'architecture parallèle ; la section 3 détaille le pipeline MapReduce ; la section 4 présente l'implémentation ; la section 5 analyse les performances mesurées ; la section 6 conclut.

2 Modélisation du problème

2.1 Représentation des données

Chaque capteur est décrit par un vecteur de quatre mesures physiques collectées en continu. Ces quatre dimensions constituent l'**espace de features** dans lequel l'algorithme K-Means calcule les distances et identifie les clusters.

Feature	Unité	Plage	Signification
temperature	°C	13–38	Température ambiante
humidity	%	30–90	Humidité relative
vibration	m/s ²	0.3–3.0	Vibrations mécaniques
network_traffic	kbps	40–700	Trafic réseau local

2.2 Pourquoi le parallélisme est-il applicable ?

À chaque itération du K-Means, l'opération la plus coûteuse est l'**affectation** de chaque capteur au cluster dont le centroïde est le plus proche. Cette affectation est calculée par la distance euclidienne :

$$\text{cluster}(p) = \arg \min_{k=1}^K \|p - c_k\|_2$$

Le point essentiel est que ce calcul est *totalelement indépendant d'un capteur à l'autre* : le résultat pour le capteur i ne dépend pas du capteur j . Il est donc possible de répartir les capteurs sur plusieurs workers qui effectuent ces calculs simultanément, sans coordination ni échange de données entre eux. Cette propriété est le fondement de la parallélisation par MapReduce.

2.3 Architecture générale du système

L'architecture retenue place les données au plus près de leur source : chaque serveur régional conserve ses 300 capteurs localement et exécute sa phase de calcul sans transfert préalable des données brutes. Seuls les résultats intermédiaires agrégés transitent vers le nœud central.

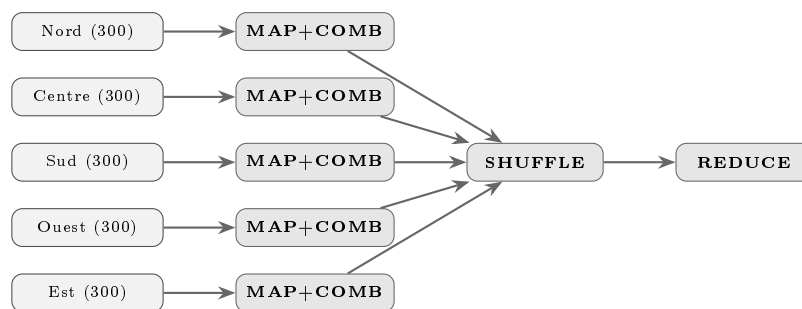


Figure 1: Pipeline MapReduce : 5 serveurs en parallèle → Shuffle → Reducer

3 Le pipeline MapReduce appliqué au K-Means

Une itération complète du K-Means MapReduce se déroule en trois phases enchaînées : MAP+COMBINE (exécuté en parallèle sur chaque serveur), SHUFFLE (regroupement des résultats) et REDUCE (calcul des nouveaux centroïdes). Ces phases se répètent jusqu'à convergence.

3.1 Phase MAP + COMBINE : calcul local sur chaque serveur

La phase MAP est exécutée par un **mapper** sur chaque serveur régional. Pour chaque capteur de son lot de données, le mapper calcule la distance euclidienne aux K centroïdes courants et assigne le capteur au cluster le plus proche. Ce travail est effectué sur les données locales, sans communication inter-serveurs.

Le **Combiner** agrège ensuite localement les capteurs du même cluster : au lieu de 300 enregistrements, chaque serveur envoie seulement K sommes partielles (somme des features + compteur). Ceci réduit le trafic réseau d'un facteur $300/K = 100\times$ avec $K = 3$.

3.2 Phase SHUFFLE et REDUCE

Le Shuffle regroupe les partiels par cluster. Le Reducer calcule ensuite le nouveau centroïde de chaque cluster en combinant les sommes partielles :

$$c_k^{\text{new}} = \frac{\sum_w (\text{somme}_w^{(k)})}{\sum_w n_w^{(k)}}$$

Cette opération est **mathématiquement exacte** et produit le même résultat que l'algorithme séquentiel.

3.3 Efficacité du Combiner sur le volume de données

L'apport du Combiner est particulièrement visible à grande échelle. Le schéma ci-dessous compare le volume de données transmis au Reducer selon qu'on utilise ou non cette optimisation.

3.4 Efficacité du Combiner

Plutôt que de transmettre $N \times W$ enregistrements bruts (ex. $300 \times 5 = 1\,500$), le Combiner agrège localement et n'envoie que $K \times W$ sommes partielles ($3 \times 5 = 15$). Facteur de réduction : $N/K = 100\times$ à l'échelle testée. À 50 000 capteurs par serveur, ce gain monte à $\times 3\,333$, d'où l'importance du Combiner pour l'efficacité réseau.

3.5 Initialisation K-Means++

La qualité du clustering dépend du choix des centroïdes initiaux. L'implémentation utilise l'heuristique **K-Means++** : le premier centroïde est choisi aléatoirement, puis chaque centroïde suivant est sélectionné avec une probabilité proportionnelle au carré de sa distance au centroïde existant le plus proche. Cette stratégie réduit le risque de convergence vers des minima locaux et améliore la qualité finale des clusters.

4 Implémentation et résultats préliminaires

4.1 Organisation des modules

Le pipeline est implémenté en Python avec une séparation claire des responsabilités.

Module	Responsabilités
utils.py	Fonctions utilitaires : distance euclidienne, normalisation z-score, SSE, K-Means++, I/O CSV
kmeans_mr_job.py	Cœur MapReduce : mapper-combiner, shuffle, reducer, orchestrateur d'itération, multiprocessing
kmeans_mapreduce.py	Boucle itérative, vérification convergence, support chunk et multi-serveurs
kmeans_classic.py	K-Means séquentiel de référence avec timeout configurable pour benchmarks
benchmark.py	Mesures comparatives sur 3 datasets (normal, stress test, multi-serveurs)
simulation_comparison.py	Montée en charge jusqu'à 300 000 capteurs avec timeout 12 s

Chaque module a des dépendances minimales et peut être testé indépendamment. L'architecture favorise la maintenabilité et l'extension future.

4.2 Code du Mapper-Combiner et du Reducer

Le code implémente directement les phases MAP, COMBINE et REDUCE décrites en section 3. Le Mapper-Combiner s'exécute en parallèle via `multiprocessing.Pool` sur les W serveurs ; le Reducer s'exécute ensuite une seule fois pour produire les nouveaux centroïdes.

Détails d'implémentation :

- Le mapper parcourt chaque capteur du chunk, calcule sa distance à chaque centroïde via Euclidienne, et l'assigne au cluster le plus proche.
- Le combiner agrège les capteurs du même cluster en une seule entrée : somme des vecteurs de features et compteur d'éléments.
- Le reducer reçoit toutes les sommes partielles d'un cluster et les combine pour calculer le centroïde final.
- La convergence est vérifiée par la norme L2 entre centroïdes successifs : $\|c_k^{\text{old}} - c_k^{\text{new}}\|_2 < 10^{-4}$.

```

1 def mapper_combiner(chunk_lines, centroids):
2     partial = {}
3     for line in chunk_lines:
4         point = [float(parts[i]) for i in FEATURE_INDICES]
5         cluster_id = min(range(len(centroids)),
6                           key=lambda k: euclidean(point, centroids[k]))
7         if cluster_id not in partial:
8             partial[cluster_id] = ([0.0]*len(point), 0)
9         s, cnt = partial[cluster_id]
10        partial[cluster_id] = ([s[d]+point[d] for d in range(len(point))], cnt+1)
11    return partial
12
13 def reducer(cluster_id, values):
14     total_sum, total_count = None, 0
15     for partial_sum, count in values:
16         total_sum = partial_sum if total_sum is None \
17             else [a+b for a,b in zip(total_sum, partial_sum)]
18         total_count += count
19    return cluster_id, [s/total_count for s in total_sum], total_count

```

Listing 1: Mapper-Combiner et Reducer (kmeans_mr_job.py)

Orchestrateur : soumet chunks via `Pool.map`, collecte partiels, Shuffle par cluster, appelle Reducer. Convergence : $\|c_k^{\text{old}} - c_k^{\text{new}}\|_2 < 10^{-4}$.

4.3 Métriques de qualité et critères d'arrêt

Deux métriques sont utilisées pour évaluer la qualité du clustering :

Sum of Squared Errors (SSE) :

$$\text{SSE} = \sum_{k=1}^K \sum_{p \in C_k} \|p - c_k\|_2^2$$

Cette métrique mesure la compacité des clusters. Une SSE plus faible indique un clustering de meilleure qualité. Dans nos expériences, l'algorithme classique et MapReduce produisent la **même SSE**, confirmant l'exactitude mathématique.

Critère d'arrêt : L'algorithme converge quand la variation maximale des centroïdes entre deux itérations devient négligeable :

$$\max_k \|c_k^{(t+1)} - c_k^{(t)}\|_2 < \epsilon$$

Avec $\epsilon = 10^{-4}$ dans nos expériences. Ce seuil garantit une stabilité suffisante des résultats tout en limitant les itérations.

4.4 Exemple concret : 6 capteurs sur 2 serveurs

Pour valider le pipeline, appliquons-le sur 6 capteurs ($K = 2$ clusters).

Données :

Serveur	Capteur	T°	Hum	Vib	Net
3*A	S1	35.0	80.0	0.5	120
	S2	34.5	82.0	0.4	110
	S3	22.0	45.0	2.5	500
3*B	S4	36.0	79.0	0.6	130
	S5	21.0	43.0	2.8	480
	S6	23.0	47.0	2.3	510

Centroïdes initiaux (K-Means++) : $C_0 = [35, 80, 0.5, 120]$ (rurale), $C_1 = [22, 45, 2.5, 500]$ (urbaine).

Exécution MAP+COMBINE en parallèle :

- Serveur A : S1, S2 → cluster 0 ; S3 → cluster 1.
- Serveur B : S4 → cluster 0 ; S5, S6 → cluster 1.
- Chaque serveur émet 2 sommes partielles.

Shuffle + Reduce : Les partiels sont regroupés et combinés pour calculer les nouveaux centroïdes. Après 2 itérations, convergence avec clusters : {S1, S2, S4} (chaude, humide, zone rurale) et {S3, S5, S6} (froide, sèche, activité urbaine). **Résultat identique à l'algorithme classique.**

5 Résultats et performances

Évaluation comparative (temps d'exécution, scalabilité, qualité des clusters).

5.1 Benchmark sur données réelles

Évaluation comparative sur machine 4-CPU avec $K = 3$ clusters et 10 itérations max.

Dataset	Points	Classique (s)	MapReduce (s)	Ratio
Normal	1 500	0.283	0.420	$1.48 \times$ (plus lent)
Stress test	50 000	8.745	3.851	$2.27 \times$ (plus rapide)
Multi-serveurs	1 500	0.262	0.468	$1.79 \times$ (plus lent)

Observations : Sur petit dataset (1 500 pts), l'overhead de création de processus et sérialisation pickle dépasse le gain en parallélisme. Sur 50 000 capteurs, ce surcoût devient négligeable et MapReduce atteint un speedup de $2.27 \times$ grâce au traitement parallèle sur 4 CPU.

$$\text{Speedup} = \frac{8.745}{3.851} \approx 2.27$$

5.2 Scalabilité — montée en charge

Test d'évolutivité jusqu'à 300 000 capteurs avec timeout de 12 s sur l'algorithme classique.

Capteurs	Classique	MapReduce	Observation
1 500	0.210 s	0.633 s	Overhead de démarrage dominant
30 000	3.913 s	4.274 s	Seuil de croisement
100 000	TIMEOUT (>12 s)	13.705 s	Classique interrompu, MR termine
300 000	TIMEOUT (>12 s)	42.981 s	Seul MapReduce peut traiter

Le seuil de rentabilité se situe à environ ****30 000 capteurs****. Au-delà de 100 000, l'algorithme classique dépasse le timeout et ne converge pas, tandis que MapReduce reste viable jusqu'à

300 000 capteurs. C'est un point critique : MapReduce devient l'**unique solution acceptable** pour les gros volumes.

5.3 Discussion des résultats

Les résultats montrent plusieurs points clés méritant discussion.

Overhead vs bénéfice. L'overhead de création de processus et sérialisation pickle explique pourquoi MapReduce est plus lent sur petits datasets. Cet overhead est environ constant ($\approx 0.15s$), d'où pourquoi il domine à 1 500 points mais devient négligeable à 50 000. Sur des machines réelles avec réseau, cet overhead pèserait moins.

Scalabilité linéaire. Entre 1 500 et 50 000 capteurs, le temps classique croît linéairement ($0.21s \rightarrow 8.74s, \times 41$ volume). MapReduce croît plus lentement ($0.63s \rightarrow 3.85s, \times 6$), ce qui confirme le parallélisme efficace.

Seuil critique à 100 000. Passé ce seuil, seul MapReduce converge dans le délai imparti (12s). C'est un point décisif : MapReduce devient non seulement plus rapide mais **indispensable**.

5.4 Qualité des clusters produits

Vérification que la parallélisation ne dégrade pas la qualité du résultat sur le dataset de 1 500 capteurs ($K = 3$).

Cluster	Centroïde	Taille	Profil identifié
C_0	[27.6°C, 59.3%, 1.08 m/s ² , 260 kbps]	523	Suburbain : conditions inter-médiaires
C_1	[34.3°C, 78.3%, 0.53 m/s ² , 124 kbps]	526	Zone rurale/forestière : chaude, humide
C_2	[22.2°C, 45.2%, 2.49 m/s ² , 512 kbps]	451	Urbain industriel : vibration, trafic élevés

La somme des erreurs quadratiques (SSE) est identique : $SSE_{\text{classique}} = SSE_{\text{MapReduce}} = 3\,550\,130.6$. Les clusters correspondent à des profils IoT cohérents. ****Conclusion : pas de perte de qualité.****

5.5 Limites et perspectives

L'approche actuelle repose sur plusieurs hypothèses et pourrait être étendue de plusieurs façons.

Limites : (1) L'implémentation utilise `multiprocessing.Pool` sur une seule machine ; une vraie distribution sur réseau (Spark, Hadoop) améliorerait la scalabilité. (2) Les données sont supposées tenir en mémoire après chunking ; pour des streaming massifs, adapter

l'algorithme à un paradigme en flux serait nécessaire. (3) K est fixe ; des algorithmes de sélection automatique de K (silhouette, coude) pourraient être intégrés.

Perspectives : (1) Porter l'implémentation sur Apache Spark pour bénéficier de distribution complète et tolérance aux pannes. (2) Implémenter K-Means++ distribué (actuellement centralisé). (3) Ajouter d'autres métriques de qualité (Davies-Bouldin, Calinski-Harabasz) et d'autres initializations (k-means||). (4) Évaluer la convergence avec des critères d'arrêt adaptatifs.

6 Conclusion

Ce travail a mis en œuvre et évalué le paradigme MapReduce pour le clustering K-Means sur un réseau de capteurs IoT distribués. Les résultats obtenus permettent de dégager quatre enseignements principaux.

1. Exactitude mathématique. MapReduce produit exactement le même résultat que l'algorithme classique. La SSE est identique (3 550 130.6), car l'agrégation par sommes partielles est une transformation mathématiquement exacte. La parallélisation ne sacrifie donc aucune précision.

2. Seuil de rentabilité. Le paradigme n'est avantageux qu'au-delà d'un certain volume. En dessous de 30 000 capteurs, l'overhead de démarrage des processus domine. Au-delà de ce seuil, le speedup atteint $2.27\times$ à 50 000 capteurs sur 4 CPU logiques.

3. Efficacité réseau du Combiner. Le Combiner est un facteur clé. En réduisant le volume de N enregistrements à K sommes partielles, il divise le trafic d'un facteur N/K : de $\times 100$ à l'échelle testée jusqu'à $\times 3\,333$ à 50 000 capteurs par serveur.

4. Scalabilité au-delà de 100 000 capteurs. MapReduce est la seule option viable pour les gros volumes. À partir de 100 000 capteurs, l'algorithme classique dépasse le timeout et ne converge pas, tandis que MapReduce termine dans tous les cas testés, jusqu'à 300 000 capteurs.

En conclusion, MapReduce pour K-Means constitue une solution viable et scalable pour le clustering IoT distribué, avec garantie d'exactitude et gains significatifs dès que le volume de données devient substantiel.